

1. **Running JavaScript:** What JavaScript is and where it runs · The console: `console.log` and `console.assert` · Reading error messages
2. **Variables and scope:** Declaring variables: `let`, `const` and `var` · Scope, hoisting and the temporal dead zone
3. **Types and values:** Data types and `typeof` · Strings and template literals · Numbers, arithmetic and the remainder operator · Type coercion and `==` vs `===` · Truthy and falsy values
4. **Logic and control flow:** Logical operators and short-circuiting · Conditionals: `if/else`, the ternary and `switch` · Loops: `for`, `while`, `break` and `continue` · `for...of`, `for...in` and looping over strings
5. **Arrays and objects:** Arrays: indexes, length, push and pop · Array methods: `forEach`, `map`, `filter`, `find`, `includes` · Objects: properties, dot and bracket notation · Values vs references: copying and mutation
6. **Functions:** Functions: parameters, arguments and return · Declarations, expressions and arrow functions · Default parameters · Function scope and functions as values

Running JavaScript

What JavaScript is and where it runs

JavaScript is the programming language of the web: browsers run it to make pages react to the user, and Node.js runs the same language outside the browser.

- Three common ways to run JavaScript while learning: the **browser console** (F12 → Console), a `<script>` tag in a page, and `node file.js` in a terminal (or the VS Code Code Runner button).
- JavaScript is **not** Java: the names are similar for marketing reasons only.
- Every statement can end with `;`. JavaScript inserts missing semicolons automatically, but the code on this site always writes them, which avoids a few surprising edge cases.

```
console.log('Hello from JavaScript');
console.log(2 + 3);
console.log(typeof 42);
console.log(typeof document); // no page here, like Node
```

Example. Type `2 + 3` in the browser console and it answers 5; save `console.log(2 + 3);` in `sum.js` and run `node sum.js`: the terminal prints 5. Now try `typeof document`: the browser console says `'object'`, Node says `'undefined'`. Same language, different environment. The Try-it box below runs in a background worker with no page, so it behaves like Node here.

Common mistake. Assuming `document` exists everywhere. Running a DOM script with `node app.js` stops with `ReferenceError: document is not defined`: the code is fine, but Node has no page to give it.

The console: `console.log` and `console.assert`

The console is the developer's output window: `console.log` prints values, and `console.assert` prints a message only when a check fails.

```
function double(n) {
  return n * 2;
}

console.log('double(4) is', double(4));    // double(4) is 8

console.assert(double(4) === 8, 'double 4'); // silent: passed
console.assert(double(0) === 1, 'double 0'); // Assertion failed: double 0
```

```
const price = 20;
const quantity = 3;
console.log('price:', price, 'quantity:', quantity);
console.log('total', price * quantity);
console.log(['a', 'b'], { item: 'pen', price: 2 });
console.error('errors print in red');
```

Example. You write `celsiusToF(c)` and want to be sure it works. Add three checks under it:

```
console.assert(celsiusToF(0) === 32, 'freezing'), console.assert(celsiusToF(100) === 212, 'boiling'),
console.assert(celsiusToF(-40) === -40, 'same in both').
```

Reload: an empty console means all three passed; Assertion failed: boiling tells you exactly which case to fix.

Common mistake. Thinking that printing a value is the same as producing it. `console.log` shows a value **to you**; it does not give the value back to the code that called the function. A function that only logs its result returns undefined (see "Functions: parameters, arguments and return").

Reading error messages

An error message names the kind of problem, describes it and points to the file, line and column where JavaScript had to stop.

The three errors you will meet most (messages as Chrome and Node word them)

Type	Meaning	Typical message	Typical cause
SyntaxError	The code cannot even be read; nothing in the file runs	Unexpected token '}'	A missing or extra bracket, quote or comma
ReferenceError	A name does not exist here	totl is not defined	A typo, a variable used outside its scope, or a <code>let/const</code> used before its line
TypeError	The value exists but is the wrong kind for the operation	Cannot read properties of undefined (reading 'name'), x is not a function, Assignment to constant variable.	Reading a property of undefined/null, calling something that is not a function, reassigning a <code>const</code>

```
try {
  const user = undefined;
  console.log(user.name);
} catch (err) {
  console.log(err.name); // TypeError
}

try {
  console.log(totl); // typo for total
} catch (err) {
  console.log(err.name); // ReferenceError
}

const list = [];
console.log(0 / list.length); // NaN: no error, but a logic bug
```

Example. A function `average(list)` is tested with `[]` and prints `NaN` instead of `0`. There is no error, so it is a logic bug. `NaN` is the clue: it comes from `0 / 0`, so the sum (0) was divided by the length (0). The fix is to handle the empty list before dividing.

Common mistake. Fixing the line the error points at when the real cause is upstream. `TypeError: Cannot read properties of null on line 20` usually means the variable became `null` earlier, for example because `document.querySelector('#sav')` had a typo and found nothing. Also: fix the **first** error in the console first; later ones are often consequences of it.

Variables and scope

Declaring variables: `let`, `const` and `var`

A variable is a name bound to a value: a `const` name can never be pointed at another value, a `let` name can, and `var` is the older keyword with looser rules.

The three keywords side by side

Keyword	Reassign?	Scope	Used before its line	Declare the same name twice
<code>const</code>	No	Block { }	ReferenceError	SyntaxError
<code>let</code>	Yes	Block { }	ReferenceError	SyntaxError

Keyword	Reassign?	Scope	Used before its line	Declare the same name twice
var	Yes	Whole function (ignores blocks)	Silently undefined	Allowed, silently

```

let count = 0;
count = count + 1;
console.log(count);           // 1

const city = 'Madrid';
try {
  city = 'Bilbao';
} catch (err) {
  console.log(err.name);      // TypeError
}

const items = [];
items.push('pen');           // allowed: changes the array
console.log(items);

```

Example. A shopping cart: `const TAX_RATE = 0.21`; never changes, so `const`. `let total = 0`; grows as items are added (`total = total + price`);, so `let`. `const items = []`; is also `const`: you will add items **into** the array, but the name `items` always points at the same array.

Common mistake. "const means the value can never change." It only freezes the **binding** (which value the name points at). `const items = []`; `items.push('pen')`; is legal, because the array is changed, not replaced. See "Values vs references".

Scope, hoisting and the temporal dead zone

Scope is the region of code where a name is visible; **hoisting** is the engine registering every declaration before it runs a scope, so `var` reads as `undefined` early while `let` and `const` throw an error.

- **Shadowing**: an inner `let x` hides an outer `x` inside its block; the outer one is untouched.
- A `for (let i = ...) loop` gets a fresh `i` per pass that disappears after the loop; with `var i` the counter survives the loop.
- Why the TDZ is a good thing: a bug that would silently give `undefined` with `var` becomes a loud error with `let/const`.

```

console.log(a);           // undefined: var is hoisted with undefined
var a = 1;

try {
  console.log(b);         // b is in its temporal dead zone
} catch (err) {
  console.log(err.name);  // ReferenceError
}
let b = 2;

if (true) {
  var leaky = 'var escapes the block';
  let tidy = 'let stays inside';
}
console.log(leaky);
console.log(typeof tidy); // 'undefined': not visible out here

for (var i = 0; i < 3; i++) {}
console.log(i);           // 3: the var counter survived the loop

```

Example. Shadowing plus TDZ: `let x = 'outer'; { console.log(x); let x = 'inner'; }` throws `ReferenceError` instead of printing `outer`. The inner `let x` was hoisted to the top of its block and is in its TDZ at the `console.log` line, which proves `let` **is** hoisted, just locked.

Common mistake. "let and const are not hoisted." They are; they are just unusable until their line. The visible difference with `var` is the error versus the silent `undefined`, not whether hoisting happens.

Types and values

Data types and typeof

Every value has a type: JavaScript has seven primitive types plus objects, and the `typeof` operator tells you which one a value is, with two famous quirks.

What `typeof` answers

Value	typeof value	Note
42, 3.14, NaN	'number'	One type for integers and decimals; even "Not a Number" is a number
'hi', ""	'string'	
true	'boolean'	
undefined	'undefined'	Also for a name that was never declared
null	'object'	A historical bug kept for compatibility: test with <code>value === null</code>
[1, 2], { a: 1 }	'object'	Arrays are objects: test with <code>Array.isArray(value)</code>
function () {}	'function'	Functions are objects too, but <code>typeof</code> reports them separately

```
console.log(typeof 42, typeof 'hi', typeof true);
console.log(typeof undefined, typeof null);
console.log(typeof [1, 2], Array.isArray([1, 2]));
console.log(typeof function () {});
```

```
let x = 5;
x = 'five';           // the value decides the type
console.log(typeof x);
```

Example. The value of a form field is **always a string**: if the user types 42 into an age box, `typeof input.value` is 'string' and `input.value + 1` is '421'. Converting with `Number(input.value)` gives the number 42.

Common mistake. Testing `typeof x === 'object'` to mean "x is a real object". `null` passes that test too, and the next line `x.name` then crashes with a `TypeError`. Check `x !== null` as well.

Strings and template literals

A string is an immutable sequence of characters; **template literals (backticks)** can embed expressions with `${...}` and span several lines.

String tools you will use most

Expression	Result for <code>s = 'Riverside FC'</code>	What it does
<code>s.length</code>	12	Number of characters (a property, no parentheses)
<code>s[0], s[s.length - 1]</code>	'R', 'C'	One character by index
<code>s.toLowerCase() / s.toUpperCase()</code>	'riverside fc' / 'RIVERSIDE FC'	Case conversion (new string)
<code>s.includes('FC')</code>	true	Does it contain this text? (case-sensitive)
<code>s.indexOf('z')</code>	-1	Position of the first match, -1 when absent
<code>s.slice(0, 9)</code>	'Riverside'	Characters from index 0 up to (not including) 9
<code>' hi '.trim()</code>	'hi'	Removes spaces at both ends (useful on user input)
<code>'a,b,c'.split(',')</code>	['a', 'b', 'c']	Cuts a string into an array
<code>['a', 'b'].join('--')</code>	'a-b'	The opposite: glues an array into a string

```
const club = 'Riverside FC';
console.log(club.length);
console.log(club[0], club.toUpperCase());
console.log(club.includes('FC'), club.indexOf('z'));
console.log(club.slice(0, 9));
console.log('a,b,c'.split(','));

const goals = 3;
console.log(`${club} scored ${goals} goal${goals === 1 ? '' : 's'}`);
```

```
club.toLowerCase();      // result thrown away...
console.log(club);        // ...so club is unchanged
```

Example. Building a score line: the template literal `${home} ${homeGoals}-${awayGoals} ${away}`, written between backticks, with `home = 'Riverside'`, `homeGoals = 2`, `awayGoals = 1`, `away = 'Hillside'` gives `Riverside 2-1 Hillside`. Any expression fits inside `${...}`, even `${homeGoals > awayGoals ? 'win' : 'no win'}`.

Common mistake. Expecting a method to change the string: after `club.toUpperCase()`; the variable `club` is unchanged. Strings are immutable, so keep the result: `const loud = club.toUpperCase()`. A second classic: writing `'Hello ${user}'` with normal quotes, which prints the `${user}` literally; interpolation only works between backticks.

Numbers, arithmetic and the remainder operator

JavaScript has a single number type for integers and decimals; besides `+` `-` `*` `/` it has `%` (remainder) and `**` (power), and decimals are stored in binary, so some results are slightly off.

- Shorthand: `x += 5` means `x = x + 5` (also `-=`, `*=`, `/=`); `x++` adds 1, `x--` subtracts 1.
- `Math.round(x)`, `Math.floor(x)` (down), `Math.ceil(x)` (up), `Math.sqrt(x)`, `Math.abs(x)`. Round to cents: `Math.round(x * 100) / 100`. `x.toFixed(2)` also rounds, but returns a **string**.
- `Number('42') → 42`; `Number('abc') → NaN` ("Not a Number", the result of failed maths). `NaN` is not equal to anything, not even itself: test it with `Number.isNaN(x)`.
- Dividing by zero does not crash: `1 / 0` is Infinity.

```
console.log(17 % 5);          // 2
console.log(10 % 2 === 0);    // true: 10 is even
console.log(0.1 + 0.2);
console.log(Math.round(12.3456 * 100) / 100);
console.log(Number('42') + 1, Number('abc'));
console.log(Number('abc') === NaN, Number.isNaN(Number('abc')));

let score = 10;
score += 5;
score++;
console.log(score);           // 16
```

Example. Converting 135 minutes into hours and minutes: `Math.floor(135 / 60)` is 2 whole hours and `135 % 60` is 15 minutes left over, so for `m = 135` the template literal `${Math.floor(m / 60)} h ${m % 60} min` (between backticks) prints `2 h 15 min`.

Common mistake. Testing `x === NaN` to detect a failed conversion. It is **always** false, even when `x` is `NaN`, because `NaN` never equals anything. Use `Number.isNaN(x)`.

Type coercion and `==` vs `===`

Coercion is JavaScript converting a value to another type automatically; `===` compares type and value without converting, while `==` converts first, so the safe default (and the one used on this site) is `===`.

Loose versus strict equality

Comparison	<code>==</code>	<code>===</code>
<code>'5'</code> and <code>5</code>	true	false
<code>0</code> and <code>''</code>	true	false
<code>0</code> and <code>false</code>	true	false
<code>null</code> and <code>undefined</code>	true	false
<code>''</code> and <code>'0'</code>	false	false
<code>NaN</code> and <code>NaN</code>	false	false

```
console.log('5' + 1);          // '51': + joins text
console.log('5' - 1);          // 4: - converts to number
console.log('5' == 5, '5' === 5);
console.log(0 == '', 0 === '');
console.log(null == undefined, null === undefined);
```

```
console.log(NaN === NaN);
console.log([1] === [1]);           // two different arrays
```

Example. An age gate reads `const raw = '18'`; from a form. `raw >= 18` is true (coerced), `raw === 18` is false (string vs number), and `Number(raw) === 18` is true. Converting once, at the point where the input enters your code, makes every later comparison predictable.

Common mistake. Comparing form input with a number: `if (input.value === 18)` is never true because `input.value` is the string `'18'`. Switching to `==` makes it "work" but hides the real problem; convert with `Number()` instead.

Truthy and falsy values

In a condition every value counts as true or false: the falsy values are `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined` and `NaN`; everything else is truthy.

```
const cases = [
  ['0', 0], ['"', ''], ['"', ''], ['"', ''], ['"', ''],
  ['null', null], ['undefined', undefined], ['NaN', NaN],
  ['[]', []], ['{}', {}], ['-1', -1],
];
for (const pair of cases) {
  console.log(pair[0], Boolean(pair[1]) ? 'truthy' : 'falsy');
}
```

Example. A search box: `const query = input.value.trim(); if (!query) { showMessage('Type something'); }`. An empty or all-spaces input becomes `''`, which is falsy, so one test covers both cases.

Common mistake. Using truthiness when `0` or `''` is a valid value: `if (stock) { ... } else { show('no data') }` shows "no data" when the stock is exactly 0. Test the case you really mean, e.g. `stock !== undefined`. The opposite trap: `if (results)` is always true for an array, even an empty one; test `results.length`.

Logic and control flow

Logical operators and short-circuiting

`&&`, `||` and `??` combine values and stop as soon as the answer is known, returning one of their operands; `!` turns any value into the opposite boolean.

- Optional chaining `a?.b` is the modern guard: it gives `undefined` instead of crashing when `a` is `null` or `undefined`.

Defaults: `||` versus `??`

Expression	Result	Why
<code>'' 'guest'</code>	<code>'guest'</code>	<code>''</code> is falsy
<code>'' ?? 'guest'</code>	<code>''</code>	<code>''</code> is not null/undefined
<code>0 50</code>	<code>50</code>	<code>0</code> is falsy
<code>0 ?? 50</code>	<code>0</code>	<code>0</code> is a real value
<code>undefined ?? 50</code>	<code>50</code>	No value at all

```
console.log(true && 'yes');           // 'yes'
console.log(0 && 'never');             // 0
console.log('' || 'guest');           // 'guest'
console.log(0 || 50, 0 ?? 50);        // 50 0
console.log(null ?? 'default');
console.log(!'hello', !!'hello');

function shout() {
  console.log('shout ran');
  return true;
}
console.log(false && shout());          // shout never runs
const user = undefined;
console.log(user && user.name);         // undefined, no crash
```

Example. A greeting: `const display = nickname || fullName || 'guest';` picks the first non-empty name. A volume setting: `const volume = settings.volume ?? 50;` keeps a deliberate volume of 0, which `||` would wrongly replace by 50.

Common mistake. Using `||` for a default when 0, '' or false is a legitimate value. `const volume = settings.volume || 50` turns a muted player (volume 0) into volume 50. Use `??` when only "missing" should trigger the default.

Conditionals: if/else, the ternary and switch

Conditionals choose which code runs: `if / else if / else` for general branching, the ternary `cond ? a : b` to pick one of two values, and `switch` to compare one value against fixed cases.

- Comparison operators: `===`, `!==`, `<`, `>`, `<=`, `>=`. A single `=` is assignment, not comparison: `if (x = 5)` assigns 5 and is always truthy.
- A **guard clause** (an early return at the top of a function for the bad case) avoids deep nesting.

```
const score = 7.5;
if (score >= 9) {
  console.log('Outstanding');
} else if (score >= 5) {
  console.log('Pass');
} else {
  console.log('Fail');
}

const label = score >= 5 ? 'pass' : 'fail';
console.log(label);

const day = 'Sun';
switch (day) {
  case 'Sat':
  case 'Sun':
    console.log('weekend');
    break;
  default:
    console.log('weekday');
}

const level = '2';
switch (level) {
  case 2:
    console.log('number 2');
    break;
  default:
    console.log('no match: switch uses ===');
}
```

Example. Exam labels: `if (score >= 9) label = 'Outstanding'; else if (score >= 5) label = 'Pass'; else label = 'Fail';`. With `score = 7.5` the first test fails, the second passes, and the `else` is never looked at.

Common mistake. Ordering branches from general to specific: `if (score >= 5) { ... } else if (score >= 9) { ... }`. A 9.5 already matches the first test, so the "Outstanding" branch can never run.

Loops: for, while, break and continue

A loop repeats a block while a condition stays truthy; `for` puts the start, the condition and the step on one line, `while` only has the condition.

- **Accumulator pattern:** create the result before the loop (`let total = 0` or `const found = []`), update it inside, use it after the loop.
- **Off-by-one:** `<` versus `<=` decides whether the last value is included. Check the first and last pass explicitly.

```
for (let i = 0; i < 3; i++) {
  console.log('pass', i);
}
```

```

let n = 40;
let steps = 0;
while (n > 1) {
  n = Math.floor(n / 2);
  steps++;
}
console.log('halvings:', steps);

for (let i = 1; i <= 10; i++) {
  if (i % 3 !== 0) continue; // skip non-multiples
  if (i > 7) break;         // stop completely
  console.log('multiple of 3:', i);
}

```

Example. How many times can you halve 40 before reaching 1? `let n = 40, steps = 0; while (n > 1) { n = Math.floor(n / 2); steps++; }`. Trace: $40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1$, so steps ends at 5. You could not know "5" before running, which is why while fits.

Common mistake. Writing `i <= arr.length` to walk an array. The indexes go from 0 to `arr.length - 1`, so the last pass reads `arr[arr.length]`, which is undefined. Use `i < arr.length` (or `for...of`).

for...of, for...in and looping over strings

`for...of` **walks the values of an array or the characters of a string**; `for...in` **walks the keys of an object and should not be used on arrays**.

Which loop for which job

You need...	Use
Every value of an array or every character of a string	<code>for (const x of arr)</code>
The index as well, or a custom step	<code>for (let i = 0; i < arr.length; i++)</code>
Every key of an object	<code>for (const key in obj)</code> or <code>Object.keys(obj)</code>
Repeat until something happens	<code>while (condition)</code>

```

const tags = ['html', 'css', 'js'];
for (const tag of tags) {
  console.log('value:', tag);
}
for (const i in tags) {
  console.log('key:', i, typeof i);
}

const player = { name: 'Sofia', number: 10 };
for (const key in player) {
  console.log(key, '=', player[key]);
}

for (const ch of 'hey') {
  console.log(ch);
}

try {
  for (const x of player) {}
} catch (err) {
  console.log(err.name); // TypeError: objects are not iterable
}

```

Example. Counting capital letters in 'Hello World': `let caps = 0; for (const ch of 'Hello World') { if (ch !== ch.toLowerCase()) caps++; }` gives 2. No index, no length, no off-by-one.

Common mistake. Using `for...in` on an array and doing arithmetic with the key: `for (const i in tags) console.log(i + 1)` prints 01, 11, 21, because `i` is the **string** '0' and + concatenates.

Arrays and objects

Arrays: indexes, length, push and pop

An array is an ordered list of values under one name; each value is read by its position, its **index**, which starts at 0.

```
const students = ['Anna', 'Bob', 'Joan'];
console.log(students[0], students.length);
console.log(students[students.length - 1]); // last
console.log(students[10]);                  // undefined, no error

students.push('Zoe');
const removed = students.pop();
console.log(removed, students.length);

students[1] = 'Bea';
console.log(students);

const squad = [{ name: 'Elena', pos: 'GK' }, { name: 'Sofia', pos: 'MF' }];
console.log(squad[1].name);
```

Example. A squad as an array of objects: `const squad = [{ name: 'Elena', pos: 'GK' }, { name: 'Sofia', pos: 'MF' }];`. `squad.length` is 2, `squad[1].name` is 'Sofia', and `squad.push({ name: 'Ines', pos: 'DF' })` adds a third player.

Common mistake. Reading the last element with `arr[arr.length]`. Because indexes start at 0, that position is one past the end and gives undefined. The last element is `arr[arr.length - 1]`.

Array methods: forEach, map, filter, find, includes

Array methods run a function you give them (a callback) on each element: `forEach` **does something**, `map` **transforms**, `filter` **keeps some**, `find` **returns the first match**, and `includes` **checks membership**.

What each method returns (none of them changes the original array)

Method	The callback returns...	The method returns...
<code>arr.forEach(fn)</code>	nothing useful	undefined
<code>arr.map(fn)</code>	the new value for this element	a new array , same length
<code>arr.filter(fn)</code>	true/false: keep it?	a new array , possibly shorter or empty
<code>arr.find(fn)</code>	true/false: is this the one?	the first match, or undefined
<code>arr.includes(value)</code>	(no callback)	true or false

```
const prices = [12, 45, 8, 30];

prices.forEach((p) => console.log('price', p));
console.log(prices.map((p) => p * 2));
console.log(prices.filter((p) => p < 20));
console.log(prices.find((p) => p > 40));
console.log(prices.find((p) => p > 100)); // undefined
console.log(prices.includes(8), prices.includes('8'));
console.log(prices);                    // unchanged

console.log(prices.map((p) => { p * 2; })); // missing return
```

Example. Prices [12, 45, 8, 30]: `prices.map((p) => p * 2)` gives [24, 90, 16, 60]; `prices.filter((p) => p < 20)` gives [12, 8]; `prices.find((p) => p > 40)` gives 45; `prices.includes(8)` is true but `prices.includes('8')` is false (string versus number). `prices` itself is still [12, 45, 8, 30].

Common mistake. Writing a map callback with braces but no return: `prices.map((p) => { p * 2; })` gives `[undefined, undefined, undefined, undefined]`. With braces, an arrow function needs an explicit return; without braces `((p) => p * 2)` the value is returned automatically. Similarly, `const doubled = prices.forEach(...)` is always undefined: `forEach` returns nothing.

Objects: properties, dot and bracket notation

An object groups named values, its **properties, under one variable; you read and write them with dot notation**

obj.key or bracket notation `obj['key']`.

[Array or object?](#)

	Array	Object
Look values up by	position (0, 1, 2...)	name ('title', 'price')
Best for	a list of similar items	one item with named fields
Literal	<code>[]</code>	<code>{ }</code>
Count	<code>arr.length</code>	<code>Object.keys(obj).length</code>

```
const player = {
  name: 'Sofia',
  number: 10,
  stats: { goals: 7, assists: 3 },
  celebrate() {           // method shorthand
    return 'Goal!';
  },
};

console.log(player.name, player['number']);
const field = 'assists';
console.log(player.stats[field], player.stats.field);
console.log(player.club);      // undefined: missing key

player.club = 'Riverside';
console.log(Object.keys(player));
console.log(player.celebrate());
```

Example. A player record: `const player = { name: 'Sofia', number: 10, stats: { goals: 7, assists: 3 } }`. `player.stats.goals` is 7. To read whichever stat the user chose, `const field = 'assists'; player.stats[field]` gives 3.

Common mistake. Using dot notation with a variable: `player.stats.field` looks for a property literally called "field" and gives undefined. When the key is in a variable, use brackets: `player.stats[field]`.

Values vs references: copying and mutation

Assigning a primitive copies the value itself; assigning an object or array copies a **reference to the same data, so a change made through one variable is visible through the other.**

```
let a = 5;
let b = a;
b = 6;
console.log(a, b);           // 5 6: independent

const list = ['x'];
const alias = list;          // same array
alias.push('y');
console.log(list);

const copy = [...list];      // a new array
copy.push('z');
console.log(list.length, copy.length);

console.log([1] === [1], list === alias);
```

```
const team = { name: 'A', players: ['Ana'] };
const clone = { ...team };    // shallow copy
clone.players.push('Bea');
console.log(team.players);    // nested array is shared
```

Example. A function receives a reference too. `function addBonus(scores) { scores.push(10); }` called with `const mine = [7, 8]` leaves `mine` as `[7, 8, 10]` after the call: the function changed the caller's array. Writing `return [...scores, 10];` instead returns a new array and leaves `mine` intact.

Common mistake. "I copied the array, so editing the copy is safe" after writing `const backup = list;` That copies only the reference: `backup.push(...)` changes `list` too. Use `const backup = [...list];`.

Functions

Functions: parameters, arguments and return

A function is a named, reusable block of code: it receives inputs through its **parameters**, runs, and hands one result back with `return`.

```
function area(width, height) {
  return width * height;
}

const a = area(3, 4);
console.log(a);           // 12
console.log(area(3, 4) + area(1, 2));
console.log(area(3));      // NaN: height is undefined

function logOnly(x) {
  console.log('inside:', x * 2);
}
const r = logOnly(5);
console.log('returned:', r); // undefined

console.log(typeof area);    // without () you get the function itself
```

Example. `function area(width, height) { return width * height; }`. `const a = area(3, 4);` stores 12; `area(3, 4) + area(1, 2)` is 14, because each call is replaced by its returned value. `area(3)` gives NaN: height is undefined and `3 * undefined` is not a number.

Common mistake. Printing instead of returning. `function total(a, b) { console.log(a + b); }` shows the right number in the console, yet `total(2, 3)` **is** undefined, so `total(2, 3) * 2` is NaN and any test of its result fails. Return the value; let the caller decide whether to print it.

Declarations, expressions and arrow functions

A function can be written as a **declaration** (`function f() {}`), as an **expression stored in a variable** (`const f = function () {}`), or as an **arrow function** (`const f = () => ...`).

The three forms of the same function

Form	Syntax	Callable before its line?	Typical use
Declaration	<code>function add(a, b) { return a + b; }</code>	Yes (hoisted)	Named helpers at the top level of a file
Expression	<code>const add = function (a, b) { return a + b; };</code>	No: ReferenceError	Storing or passing a function as a value
Arrow	<code>const add = (a, b) => a + b;</code>	No: ReferenceError	Short helpers and callbacks (<code>map</code> , <code>addEventListener</code>)

```
console.log(early(2));    // works: declarations are hoisted
function early(x) {
  return x + 1;
}
```

```
try {
  late(2);
} catch (err) {
  console.log(err.name); // ReferenceError: const not ready yet
}

const late = function (x) {
  return x + 1;
};

const square = (n) => n * n;
const squareBraces = (n) => { n * n; };
console.log(square(4), squareBraces(4));

const makeUser = (name) => ({ name: name, active: true });
console.log(makeUser('Ana'));
```

Example. `const toEuros = (cents) => cents / 100;` (concise body, implicit return) and `const toEuros = (cents) => { return cents / 100; };` (block body) are equivalent; `toEuros(1250)` is 12.5 in both.

Common mistake. Adding braces to an arrow function and forgetting return: `const square = (n) => { n * n; };` makes `square(4)` return undefined. Either remove the braces or write `return n * n;`.

Default parameters

A default parameter (function `f(x = 10)`) supplies the value to use when the caller passes nothing, or passes undefined, for that parameter.

```
function shippingCost(weightKg, ratePerKg = 4) {
  return weightKg * ratePerKg;
}

console.log(shippingCost(2)); // 8
console.log(shippingCost(2, 6)); // 12
console.log(shippingCost(2, undefined)); // 8
console.log(shippingCost(2, null)); // 0
console.log(shippingCost(2, 0)); // 0: free shipping kept

function box(w, h = w) {
  return w + ' x ' + h;
}

console.log(box(3), box(3, 5));
```

Example. Shipping: `function shippingCost(weightKg, ratePerKg = 4) { return weightKg * ratePerKg; }.` `shippingCost(2)` is 8 (default used), `shippingCost(2, 6)` is 12, `shippingCost(2, undefined)` is 8, and `shippingCost(2, null)` is 0, because `null` is a value and `2 * null` is 0.

Common mistake. Using the old pattern `rate = rate || 4` inside the function. It also replaces a legitimate 0 (free shipping) by 4. A default parameter only replaces undefined.

Function scope and functions as values

Variables and parameters declared inside a function exist only inside it, for the duration of each call; and functions are values that can be stored, passed to other functions and returned.

- Prefer return over changing outer variables from inside a function: it keeps functions pure and testable.
- Pass the function **name** as a callback, without parentheses: `btn.addEventListener('click', save).` `save()` would call it immediately and pass its result instead.

```
const rate = 2;

function scale(x) {
  const result = x * rate; // reads the outer rate
  return result;
}

console.log(scale(5)); // 10
console.log(typeof result); // 'undefined': local to scale
```

```
function applyTwice(fn, value) {  
  return fn(fn(value));  
}  
  
console.log(applyTwice((n) => n + 3, 1));  
console.log(applyTwice((s) => s + '!', 'hi'));  
  
const tools = [Math.floor, Math.ceil];  
console.log(tools[1](2.1));    // a function stored in an array
```

Example. `function applyTwice(fn, value) { return fn(fn(value)); }` receives a function as its first argument. `applyTwice((n) => n + 3, 1)` computes $(1 + 3) + 3 = 7$; `applyTwice((s) => s + '!', 'hi')` gives 'hi!!'.

Common mistake. Expecting to read a function's local variable from outside: after function `scale(x) { const result = x * 2; return result; }`, writing `console.log(result)` at the top level is a `ReferenceError`. Use the returned value: `const r = scale(5);`.